

**UNIVERSIDAD NACIONAL DE SAN ANTONIO ABAD DEL
CUSCO**

**FACULTAD DE INGENIERÍA ELÉCTRICA, ELECTRÓNICA,
INFORMÁTICA Y MECÁNICA**

**ESCUELA PROFESIONAL DE INGENIERÍA INFORMÁTICA Y
DE SISTEMAS**

“PRACTICA DE LABORATORIO 2-DART”



Curso: Desarrollo de Software II

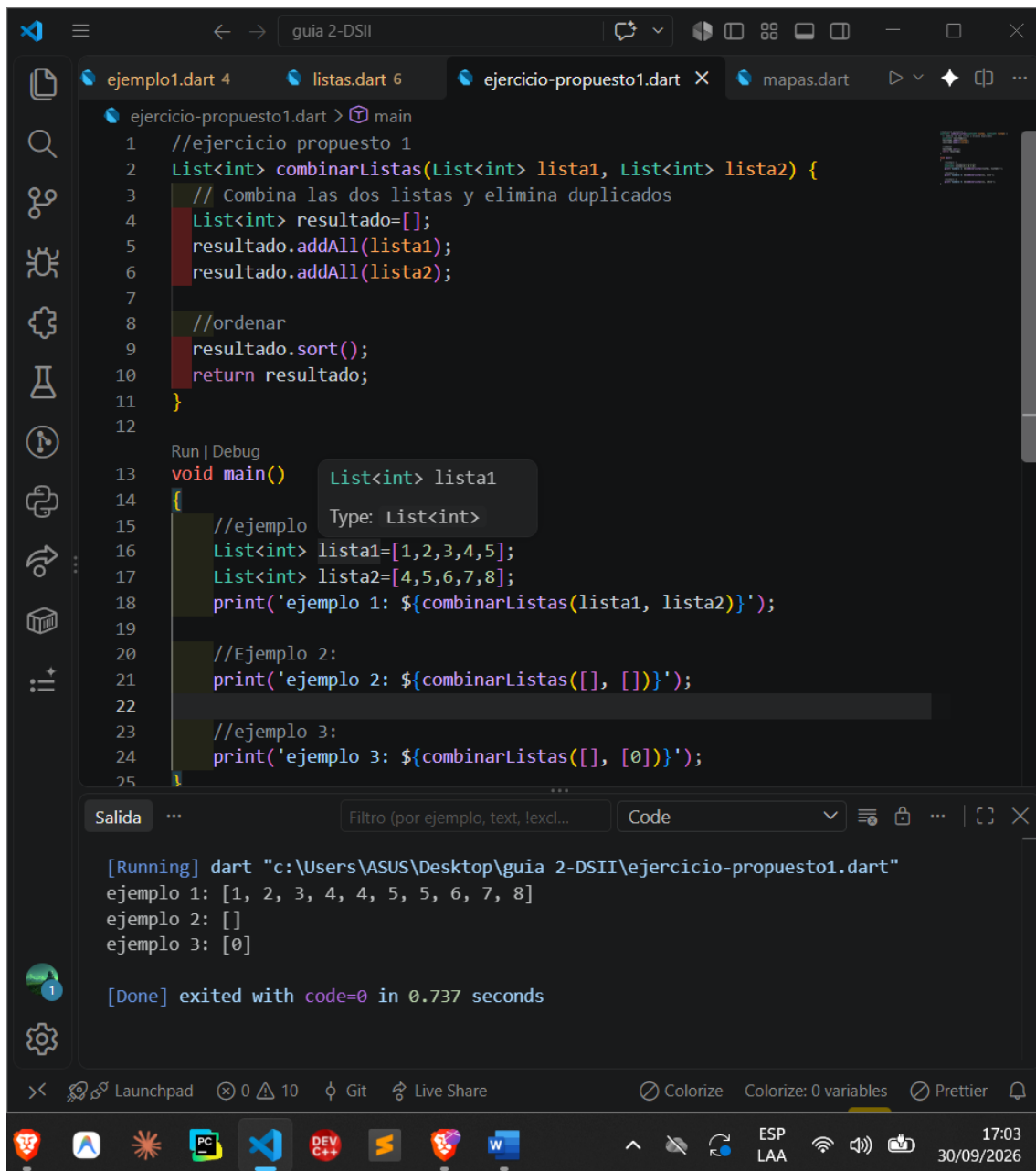
Docente: Ccacyahuillca Bejar, Hans Harley

Estudiante: Luicho Quispe, Jhoel Alex

Código: 224871

CUSCO - PERÚ

Ejercicio propuesto 1:



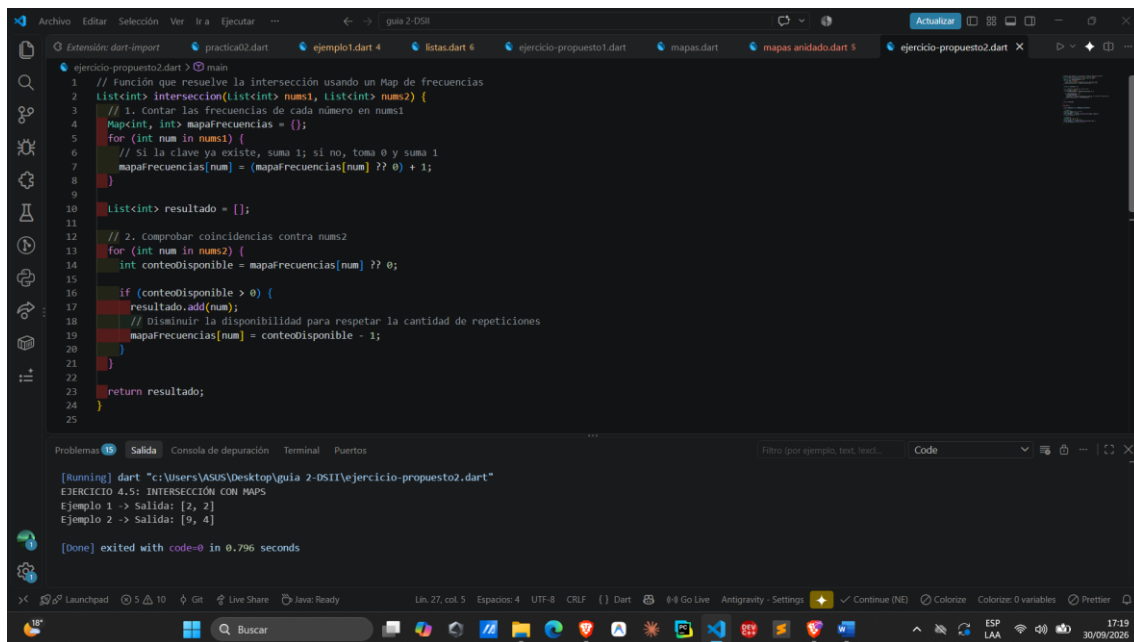
```
ejercicio-propuesto1.dart > main
1 //ejercicio propuesto 1
2 List<int> combinarListas(List<int> lista1, List<int> lista2) {
3     // Combina las dos listas y elimina duplicados
4     List<int> resultado=[];
5     resultado.addAll(lista1);
6     resultado.addAll(lista2);
7
8     //ordenar
9     resultado.sort();
10    return resultado;
11 }
12
13 void main()
14 {
15     //ejemplo
16     List<int> lista1=[1,2,3,4,5];
17     List<int> lista2=[4,5,6,7,8];
18     print('ejemplo 1: ${combinarListas(lista1, lista2)}');
19
20     //Ejemplo 2:
21     print('ejemplo 2: ${combinarListas([], [])}');
22
23     //ejemplo 3:
24     print('ejemplo 3: ${combinarListas([], [0])}');
25 }
```

Salida

```
[Running] dart "c:\Users\ASUS\Desktop\guia 2-DSII\ejercicio-propuesto1.dart"
ejemplo 1: [1, 2, 3, 4, 4, 5, 5, 6, 7, 8]
ejemplo 2: []
ejemplo 3: [0]

[Done] exited with code=0 in 0.737 seconds
```

EJERCICIO PROPUESTO 2:

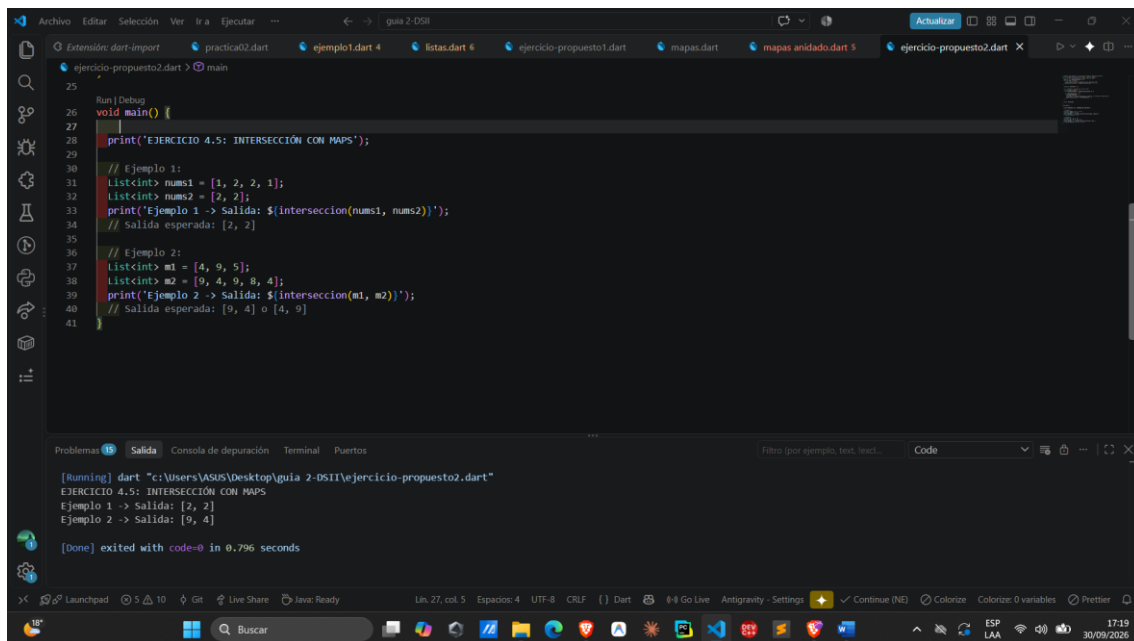


The screenshot shows the VS Code editor with the file `ejercicio-propuesto2.dart` open. The code implements a function `interseccion` that takes two lists of integers and returns their intersection. It uses a `Map` to count the frequency of each number in the first list. Then, it iterates through the second list, checking if each number exists in the first list's frequency map. If it does, it adds the number to the result list and decreases its frequency. The process continues until all numbers in the second list are processed. The final result list is returned.

```
1 // Función que resuelve la intersección usando un Map de frecuencias
2 List<int> interseccion(List<int> nums1, List<int> nums2) {
3   // 1. Contar las frecuencias de cada número en nums1
4   Map<int, int> mapafrecuencias = {};
5   for (int num in nums1) {
6     // si la clave ya existe, suma 1; si no, toma 0 y suma 1
7     mapafrecuencias[num] = (mapafrecuencias[num] ?? 0) + 1;
8   }
9
10  List<int> resultado = [];
11
12  // 2. Comprobar coincidencias contra nums2
13  for (int num in nums2) {
14    int conteoDisponible = mapafrecuencias[num] ?? 0;
15
16    if (conteoDisponible > 0) {
17      resultado.add(num);
18      // Disminuir la disponibilidad para respetar la cantidad de repeticiones
19      mapafrecuencias[num] = conteoDisponible - 1;
20    }
21  }
22  return resultado;
23 }
24
25
```

The terminal output shows the execution of the program:

```
[Running] dart "c:\Users\ASUS\Desktop\guia 2-DSII\ejercicio-propuesto2.dart"
EJERCICIO 4.5: INTERSECCIÓN CON MAPS
Ejemplo 1 -> Salida: [2, 2]
Ejemplo 2 -> Salida: [9, 4]
[Done] exited with code=0 in 0.796 seconds
```



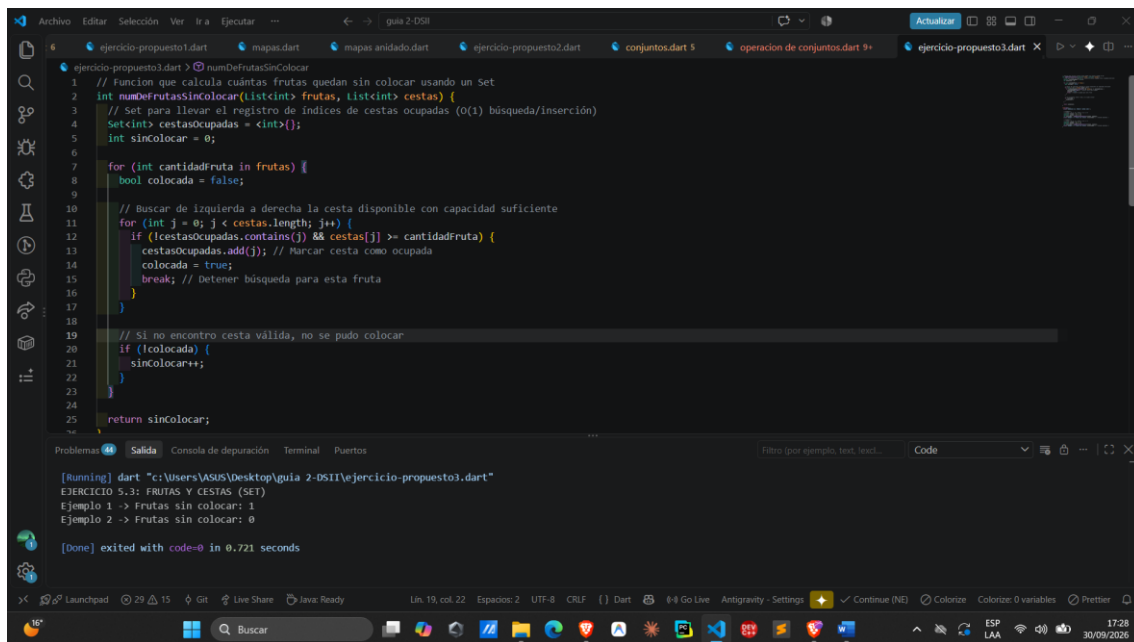
The screenshot shows the VS Code editor with the file `ejercicio-propuesto2.dart` open. The code includes test cases for the `interseccion` function. It defines two example inputs and their expected outputs, then calls the `interseccion` function with these inputs and prints the results. The terminal output shows the execution of the program, confirming that the function works as expected for the provided test cases.

```
25
26 Run | Debug
27 void main() {
28   print('EJERCICIO 4.5: INTERSECCIÓN CON MAPS');
29
30   // Ejemplo 1:
31   List<int> nums1 = [1, 2, 2, 1];
32   List<int> nums2 = [2, 2];
33   print('Ejemplo 1 -> Salida: ${interseccion(nums1, nums2)}');
34   // Salida esperada: [2, 2]
35
36   // Ejemplo 2:
37   List<int> m1 = [4, 9, 5];
38   List<int> m2 = [9, 4, 9, 8, 4];
39   print('Ejemplo 2 -> Salida: ${interseccion(m1, m2)}');
40   // Salida esperada: [9, 4] o [4, 9]
41 }
```

The terminal output shows the execution of the program:

```
[Running] dart "c:\Users\ASUS\Desktop\guia 2-DSII\ejercicio-propuesto2.dart"
EJERCICIO 4.5: INTERSECCIÓN CON MAPS
Ejemplo 1 -> Salida: [2, 2]
Ejemplo 2 -> Salida: [9, 4]
[Done] exited with code=0 in 0.796 seconds
```

EJERCICIO PROPUESTO 3:



The screenshot shows an IDE with the file `ejercicio-propuesto3.dart` open. The code implements a function `numDeFrutasSinColocar` that takes two lists of integers, `frutas` and `cestas`, and returns the number of fruits that cannot be placed in the baskets. The function uses a nested loop to check if a fruit can be placed in a basket based on its capacity and the current state of the baskets. The output window shows the results of two test cases.

```
ejercicio-propuesto3.dart > numDeFrutasSinColocar
1 // Funcion que calcula cuántas frutas quedan sin colocar usando un Set
2 int numDeFrutasSinColocar(List<int> frutas, List<int> cestas) {
3   // Set para llevar el registro de índices de cestas ocupadas (0-1) búsqueda/inserción
4   Set<int> cestasOcupadas = <int>{};
5   int sinColocar = 0;
6
7   for (int cantidadFruta in frutas) {
8     bool colocada = false;
9
10    // Buscar de izquierda a derecha la cesta disponible con capacidad suficiente
11    for (int j = 0; j < cestas.length; j++) {
12      if (!cestasOcupadas.contains(j) && cestas[j] >= cantidadFruta) {
13        cestasOcupadas.add(j); // Marcar cesta como ocupada
14        colocada = true;
15        break; // Detener búsqueda para esta fruta
16      }
17    }
18
19    // Si no encuentro cesta válida, no se pudo colocar
20    if (!colocada) {
21      sinColocar++;
22    }
23  }
24
25  return sinColocar;
26 }
```

Problemas 44 Salida Consola de depuración Terminal Puertos

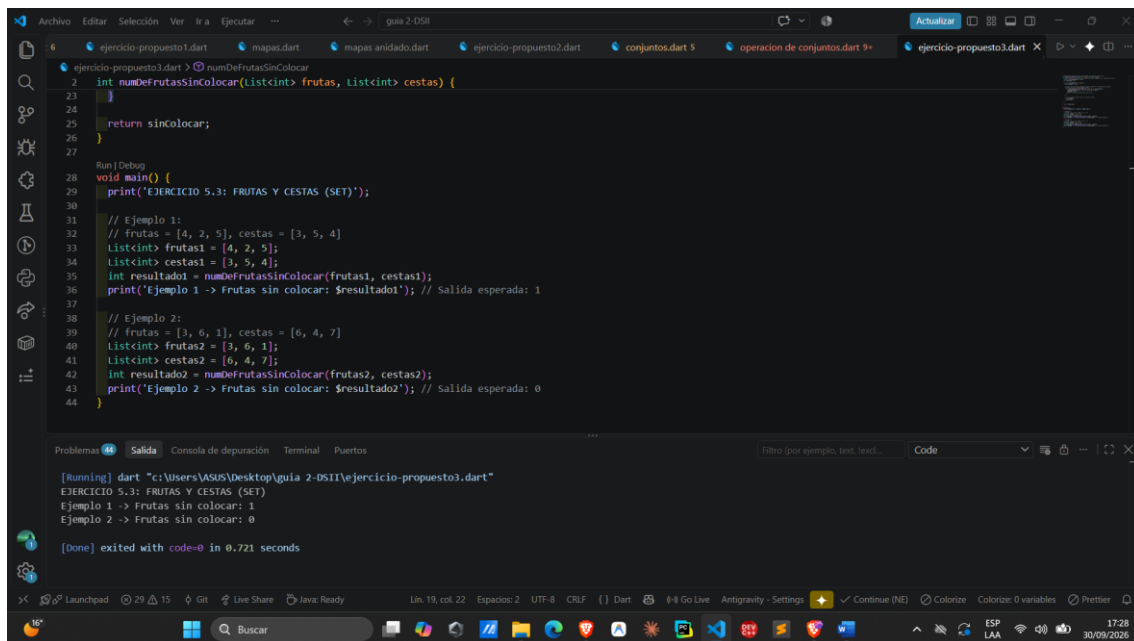
[Running] dart "c:\Users\ASUS\Desktop\guia 2-DSII\ejercicio-propuesto3.dart"

EJERCICIO 5.3: FRUTAS Y CESTAS (SET)

Ejemplo 1 -> Frutas sin colocar: 1

Ejemplo 2 -> Frutas sin colocar: 0

[Done] exited with code=0 in 0.721 seconds



The screenshot shows the same IDE with the file `ejercicio-propuesto3.dart` open. The code now includes a `main` function with two test cases. The output window shows the results of the test cases.

```
ejercicio-propuesto3.dart > numDeFrutasSinColocar
23 }
24
25 return sinColocar;
26 }
27
28 void main() {
29   print("EJERCICIO 5.3: FRUTAS Y CESTAS (SET)");
30
31   // Ejemplo 1:
32   // frutas = [4, 2, 5], cestas = [3, 5, 4]
33   List<int> frutas1 = [4, 2, 5];
34   List<int> cestas1 = [3, 5, 4];
35   int resultado1 = numDeFrutasSinColocar(frutas1, cestas1);
36   print("Ejemplo 1 -> Frutas sin colocar: $resultado1"); // Salida esperada: 1
37
38   // Ejemplo 2:
39   // frutas = [3, 6, 1], cestas = [6, 4, 7]
40   List<int> frutas2 = [3, 6, 1];
41   List<int> cestas2 = [6, 4, 7];
42   int resultado2 = numDeFrutasSinColocar(frutas2, cestas2);
43   print("Ejemplo 2 -> Frutas sin colocar: $resultado2"); // Salida esperada: 0
44 }
```

Problemas 44 Salida Consola de depuración Terminal Puertos

[Running] dart "c:\Users\ASUS\Desktop\guia 2-DSII\ejercicio-propuesto3.dart"

EJERCICIO 5.3: FRUTAS Y CESTAS (SET)

Ejemplo 1 -> Frutas sin colocar: 1

Ejemplo 2 -> Frutas sin colocar: 0

[Done] exited with code=0 in 0.721 seconds

